

泛型（类属）程序设计--基于STL的编程

1.STL (Standard Template Library)

1. 实现了一些面向序列数据的表示及常用的操作，支持了一种抽象的编程模式，该模式隐藏了一些低级的程序元素，如数组、链表、循环等。
2. 包含：
 - **容器**：用于存储序列化的数据元素，如：向量、队列、栈等。
 - **算法**：用于实现对容器中的数据元素进行一些常用的操作，如：查找、排序、统计、求和等一系列算术运算。
 - **迭代器**：实现了抽象的指针功能，它们指向容器中的数据元素，用于对容器中的数据元素进行遍历和访问。是容器和算法之间的桥梁：**传给算法的不是容器，而是指向容器中元素的迭代器，这样使得算法与容器保持独立，从而提高算法的通用性。**

2.容器

1. 容器是由同类型元素构成的、长度可变的元素序列，容器是用类模板来实现的，模板的参数是容器中元素的类型。STL中包含了很多种容器，虽然这些容器提供了一些相同的操作，但由于它们采用了不同的内部实现方法，因此，不同的容器往往适合于不同的应用场合。
2. 主要的容器：
 - vector<元素类型>
 - 用于需要快速定位（访问）任意位置上的元素以及主要在元素序列的尾部增加/删除元素的场合。在头文件vector中定义，用动态数组实现。
 - list<元素类型>
 - 用于经常在元素序列中任意位置上插入/删除元素的场合。在头文件list中定义，用双向链表实现。
 - deque<元素类型>
 - 用于主要在元素序列的两端增加/删除元素以及需要快速定位（访问）任意位置上的元素的场合。在头文件deque中定义，用分段的连续空间结构实现。
 - stack<元素类型>
 - 用于仅在元素序列的尾部增加/删除元素的场合。在头文件stack中定义，基于deque、list或vector来实现。
 - queue<元素类型>
 - 用于仅在元素序列的尾部增加、头部删除元素的场合。在头文件queue中定义，基于deque或list来实现。
 - priority_queue<元素类型>
 - 它与queue的操作类似，不同之处在于：每次增加/删除元素之后，它将对元素位置进行调整，使得头部元素总是最大的。也就是说，每次删除操作总是把最大（优先级最高）的元素去掉。
 - 在头文件queue中定义，基于deque或vector来实现。

上面三类容器是作为适配器提供的，方便使用。

- map<关键字类型，值类型>和multimap<关键字类型，值类型>

- 用于需要根据关键字来访问元素的场合。容器中每个元素属于一个pair结构类型，该结构有两个成员：first和second，关键字对应first，值对应second，元素是根据其关键字排序的。
 - 对于map，不同元素的关键字不能相同；
 - 对于multimap，不同元素的关键字可以相同。它们在头文件map中定义，常常用某种二叉树来实现。
 - set<元素类型>和multiset<元素类型>
 - 它们分别是map和multimap的特例，每个元素只有关键字而没有值，或者说，关键字与值合一了。
 - 在头文件set中定义。
 - basic_string<字符类型>
 - 与vector类似，不同之处在于其元素为字符类型，并提供了一系列与字符串相关的操作。
 - string和wstring分别是它的两个实例：


```
basic_string
basic_string<wchar_t>
```
 - 在头文件string中定义。
3. 容器的基本操作：增加、删除元素，获取指定位置上的元素，在容器中查找元素，获取容器首/尾元素的迭代器等等。
4. 如果容器的元素类型是一个类，则针对该类可能需要：
- 自定义**拷贝构造函数**和**赋值操作符重载函数**对容器进行的一些操作中可能会创建新的元素（对象的拷贝构造）或进行元素间的赋值（对象赋值）。
 - 重载小于操作符（<）对容器进行的一些操作中可能要对元素进行“小于”比较运算。

3.迭代器

- 迭代器（iterator）实现了**抽象的指针（智能指针）**功能，它们指向容器中的元素，用于**对容器中的元素进行访问和遍历**。
- 在STL中，迭代器是作为**类模板**来实现的（在头文件iterator中定义），它们可分为以下几种类型：
 - 输出迭代器（output iterator，记为：OutIt）
 - 只能修改它所指向的容器元素
 - 间接访问（*）
 - ++
 - 输入迭代器（input iterator，记为：InIt）
 - 只能读取它所指向的容器元素
 - 间接访问（）和元素成员间接访问（->）
 - ++、==、!=
 - 前向迭代器（forward iterator，记为：FwdIt）
 - 可以读取/修改它所指向的容器元素
 - 元素间接访问和元素成员间接访问（->）
 - ++、==、!=
 - 双向迭代器（bidirectional iterator，记为：BidIt）

- 可以读取/修改它所指向的容器元素
- 元素间接访问和元素成员间接访问 (->)
- ++、--、==、!=操作
- 随机访问迭代器 (random-access iterator, 记为: RanIt)
 - 可以读取/修改它所指向的容器元素
 - 元素间接访问 (*)、元素成员间接访问 (->) 和下标访问元素 ([])
 - ++、--、+、-、+=、-=、==、!=、<、>、<=、>=

注意: 指向数组元素的**普通指针**可以看成是随机访问迭代器

3. 各容器的迭代器类型:

- 对于vector、deque以及basic_string容器类, 为RanIt
- 对于list、map/multimap以及set/multiset容器类, 为BidIt
- queue、stack和priority_queue容器类, 不支持迭代器

可通过容器类的成员函数begin和end等获得容器的首尾迭代器

4. 迭代器之间的相容关系:

5. 除了上述五种迭代器, STL还提供了一些迭代器的适配器, 便于一些特殊的操作

- 反向迭代器 (reverse iterator) : 用于对容器元素从尾到头进行反向遍历: ++操作是往容器首部移动, --操作是往容器尾部移动。可以通过容器类的成员函数rbegin和rend可以获得容器的尾和首元素的反向迭代器。
- 插入迭代器 (insert iterator) : 用于在容器中指定位置插入元素, 其中包括:
 - back_insert_iterator (用于在尾部插入元素)
 - front_insert_iterator (用于在首部插入元素)
 - insert_iterator (用于在任意指定位置插入元素)
 - 它们可以分别通过全局函数back_inserter、front_inserter和inserter来获得, 函数的参数为容器。

4. 算法

1. STL算法实现了对序列元素的一些常见操作, 包括:

调序算法、编辑算法、查找算法、算术算法、集合算法、堆算法、元素遍历算法等, 除了算数算法在头文件numeric中, 其他算法都在头文件algorithm中定义。

2. 大部分STL算法都是**遍历**指定容器中**某范围**内的元素, 对满足条件的元素执行某项操作。

3. 在STL中, 不是把容器传给算法, 而是把容器的某些**迭代器**传给它们, 在算法中通过迭代器来访问和遍历相应容器中的元素。

这样做的好处是提高了算法的**通用性**: 不管什么容器, 只要它的迭代器与算法所需要的迭代器相容, 就可以传给算法, 这样就使得算法不依赖于具体的容器。需要注意的是: 有些容器 (stack、queue和priority_queue) 不支持迭代器, 因此不能用STL中的算法来操作它们!

4. 一个算法能接收的迭代器的类型是通过算法模板参数的名字来体现的。例如:

```

1  template <class InIt, class OutIt>
2  OutIt copy(InIt src_first, InIt src_last,
3             OutIt dst_first)
4  { ..... }
5  //src_first和src_last是输入迭代器，算法中只读取它们指向的元素。
6  //dst_first是输出迭代器，算法中可以修改它指向的元素。
7  //以上参数可以接受与之相容的迭代器。
8

```

5. 算法的操作范围：

用算法对容器中的元素进行操作时，大都需要用两个迭代器来指出要操作的元素的范围。如：

```

1  void sort(RanIt first, RanIt last);
2  //first是第一个元素的位置
3  //last是最后一个元素的下一个位置
4  //例如：
5  vector<int> v;
6  ..... //往容器中放了元素
7  sort(v.begin(),v.end()); //从小到大排序

```

6. 算法的自定义操作条件

- 有些算法可以让使用者提供一个函数或函数对象来作为自定义操作条件（或称为谓词），其参数类型为相应容器的元素类型，返回值为bool

自定义操作条件可分为：

Pred:一元谓词，一个参数

BinPred: 二元谓词，两个参数

- 有些算法可以让使用者提供一个函数或函数对象作为自定义操作，其参数和返回值类型由相应的算法决定

自定义操作可分为：

Oph或Fun:一元操作，一个参数

BinOp或BinFun:二元操作，两个参数